

TP Système d'Exploitation 2 : Gestion des Processus

Nom Prénom

11 décembre 2025

Table des matières

1	Introduction	2
2	Exercice 1 : Fonctions fork() et sleep()	2
2.1	Question 1 : Rôle des fonctions	2
2.2	Question 2 : Exécution avec arguments 10 20	2
2.3	Question 3 : Exécution avec arguments 10 0	3
2.4	Question 4 : attente_pere=5, attente_fils=10	3
2.5	Question 5 : Modification du code	3
3	Exercice 2 : Arbre des Processus	4
3.1	Question 1 : Commande d'affichage de l'arbre	4
3.2	Question 2 : Programme créant une arborescence	4
3.3	Question 3 : Programme affichant l'arbre courant	5
4	Exercice 3 : Gestion de la Terminaison des Processus	5
4.1	Questions 1-2 : Création et attente	5
4.2	Question 3 : Code retour = 3	6
4.3	Question 4 : kill -15 pid_fils	6
4.4	Question 5 : kill -9 pid_fils	6
5	Exercice 4 : Processus Zombies	6
5.1	Question 1 : Définition d'un zombie	6
5.2	Question 2 : Programme créant un zombie	7
5.3	Question 3 : Élimination du zombie	7
6	Exercice 5 : Comportement de exec()	7
6.1	Question 1 : Exécution normale	7
6.2	Question 2 : Avec /bin/ech	8
7	Exercice 6 : fork() et exec()	8
7.1	Question 1 : Avec execl()	8
7.2	Question 2 : Avec execv()	9
8	Exercice 7 : Exécution de ls -l /	9
9	Conclusion	10

1 Introduction

Ce TP a pour objectif d'étudier la gestion des processus sous Linux à travers plusieurs exercices pratiques. Nous aborderons notamment :

- La création de processus avec `fork()`
- La synchronisation entre processus
- La gestion de la terminaison des processus
- L'exécution de commandes externes avec `exec()`
- Les processus zombies et leur élimination

2 Exercice 1 : Fonctions `fork()` et `sleep()`

2.1 Question 1 : Rôle des fonctions

- `fork()` : Crée un nouveau processus (fils) identique au processus appelant (père).
Retourne :
 - 0 dans le processus fils
 - PID du fils dans le processus père
 - -1 en cas d'erreur
- `sleep()` : Suspend l'exécution du processus appelant pendant un nombre spécifié de secondes.

2.2 Question 2 : Exécution avec arguments 10 20

```
1 #include <unistd.h>
2 #include <sys/types.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char *argv[]) {
7     pid_t pid;
8     int attente_fils, attente_pere;
9
10    if(argc != 3) {
11        printf("usage: ex1 'attente_pere' 'attente_fils'\n");
12        exit(-1);
13    }
14
15    attente_pere = atoi(argv[1]);
16    attente_fils = atoi(argv[2]);
17
18    switch(pid = fork()) {
19        case -1:
20            perror("fork error");
21            break;
22        case 0: // Processus fils
23            sleep(attente_fils);
24            printf("fils attente finie\n");
25            break;
```

```

26         default: // Processus p re
27             sleep(attente_pere);
28             printf("pere attente finie\n");
29             break;
30     }
31     return 0;
32 }

```

Listing 1 – Code source de l'exercice 1

Résultats avec ./programme 10 20 :

- **Avant la fin du père (0-10 secondes) :**
 - PPID du père = PID du shell
 - PPID du fils = PID du père
- **Avant la fin du fils (10-20 secondes) :**
 - PPID du père = PID du shell
 - PPID du fils = 1 (init/systemd)
 - *Explication : Le fils devient orphelin après la mort du père*

2.3 Question 3 : Exécution avec arguments 10 0

Observation : Le fils termine immédiatement (sleep(0)). La commande `ps -la` montre :

- Le père toujours en cours d'exécution
- Le fils dans l'état zombie (Z)
- *Explication : Le père n'a pas appelé wait() pour récupérer le statut de terminaison du fils*

2.4 Question 4 : attente_pere=5, attente_fils=10

Ordre d'affichage :

1. "pere attente finie" après 5 secondes
2. "fils attente finie" après 10 secondes

Explication : Les processus s'exécutent concurrentiellement. Le père termine son attente en premier ($5 < 10$).

2.5 Question 5 : Modification du code

```

1 case 0:
2     sleep(attente_fils);
3     printf("fils attente finie - PID: %d, UID: %d, GID: %d\n",
4           getpid(), getuid(), getgid());
5     break;
6 default:
7     sleep(attente_pere);
8     printf("pere attente finie - PID: %d, UID: %d, GID: %d\n",
9           getpid(), getuid(), getgid());
10    break;

```

Listing 2 – Code modifié avec informations PID/UID/GID

Impact sur la sortie : Affiche en plus pour chaque processus :

- PID : Identifiant unique du processus
- UID : Identifiant de l'utilisateur propriétaire
- GID : Identifiant du groupe

3 Exercice 2 : Arbre des Processus

3.1 Question 1 : Commande d'affichage de l'arbre

```
1 # Afficher l'arbre      partir d'un PID
2 pstree -p <PID>
3
4 # Alternative avec ps
5 ps -ejf | grep <PID>
6
7 # Afficher avec indentation
8 ps f -g <PGID>
```

Listing 3 – Commandes pour afficher l'arbre des processus

3.2 Question 2 : Programme créant une arborescence

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <sys/wait.h>
5
6 int main() {
7     printf("P1: PID=%d, PPID=%d\n", getpid(), getppid());
8
9     // Cr ation de P2
10    if (fork() == 0) {
11        printf("P2: PID=%d, PPID=%d\n", getpid(), getppid());
12
13        // Cr ation de P4
14        if (fork() == 0) {
15            printf("P4: PID=%d, PPID=%d\n", getpid(), getppid());
16            exit(0);
17        }
18        wait(NULL); // P2 attend P4
19        exit(0);
20    }
21
22    // Cr ation de P3
23    if (fork() == 0) {
24        printf("P3: PID=%d, PPID=%d\n", getpid(), getppid());
25        exit(0);
26    }
27 }
```

```

28 // P1 attend P2 et P3
29 wait(NULL);
30 wait(NULL);
31
32 sleep(1);
33 return 0;
34 }

```

Listing 4 – Création d’arborescence de processus

3.3 Question 3 : Programme affichant l’arbre courant

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     printf("Arbre des processus pour PID=%d:\n", getpid());
7     system("pstree -p $$");
8     return 0;
9 }

```

Listing 5 – Affichage de l’arbre du processus courant

4 Exercice 3 : Gestion de la Terminaison des Processus

4.1 Questions 1-2 : Création et attente

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10         // Processus fils P2
11         printf("P2 (fils) - PID: %d\n", getpid());
12         exit(0); // Terminaison normale
13     } else {
14         // Processus p re P1
15         printf("P1 (p re) - PID: %d\n", getpid());
16
17         int status;
18         wait(&status); // Attente de la terminaison du fils
19
20         if (WIFEXITED(status)) {
21             printf("Fils termin avec code: %d\n",
22                 WEXITSTATUS(status));

```

```

22     }
23 }
24 return 0;
25 }

```

Listing 6 – Création et attente de processus

4.2 Question 3 : Code retour = 3

```

1 // Dans le fils :
2 exit(3);
3
4 // Dans le p re :
5 if (WIFEXITED(status)) {
6     printf("Code retour fils: %d\n", WEXITSTATUS(status)); //
7     Affiche 3
8 }

```

Listing 7 – Fils terminant avec code retour 3

4.3 Question 4 : kill -15 pid_fil

```

1 // Dans le fils :
2 sleep(10); // Long sleep
3
4 // Apr s wait(&status) dans le p re :
5 if (WIFSIGNALED(status)) {
6     printf("Fils tu par signal: %d (SIGTERM)\n",
7     WTERMSIG(status));
8 }

```

Listing 8 – Terminaison par signal

Résultat : Le père affiche "Fils tué par signal : 15" lorsque le fils est tué par kill -15.

4.4 Question 5 : kill -9 pid_fil

Résultat : Le père affiche "Fils tué par signal : 9" (SIGKILL). SIGKILL ne peut être ignoré ou capturé.

5 Exercice 4 : Processus Zombies

5.1 Question 1 : Définition d'un zombie

Un **processus zombie** est un processus qui a terminé son exécution mais dont l'entrée dans la table des processus n'a pas encore été supprimée. Cela se produit lorsque le processus parent n'a pas appelé `wait()` pour récupérer le statut de terminaison de son fils.

5.2 Question 2 : Programme créant un zombie

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         // Processus fils
10        printf("Fils (PID=%d) termin \n", getpid());
11        exit(0); // Termine immédiatement
12    } else {
13        // Processus p re
14        printf("P re (PID=%d) continue\n", getpid());
15        sleep(30); // Ne wait pas      fils devient zombie
16        printf("P re termin \n");
17    }
18    return 0;
19 }
```

Listing 9 – Création d'un processus zombie

5.3 Question 3 : Élimination du zombie

```
1 // Modifier la partie p re :
2 else {
3     printf("P re (PID=%d) continue\n", getpid());
4     sleep(5); // Temps pour observer le zombie
5     wait(NULL); // R cup re statut      limine   zombie
6     sleep(25); // Continue l'ex cution
7     printf("P re termin \n");
8 }
```

Listing 10 – Élimination du processus zombie

6 Exercice 5 : Comportement de exec()

6.1 Question 1 : Exécution normale

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main() {
6     char buf[80];
7
8     printf("[%d] - Debut du test exec()\n", getpid());
```

```

9     sprintf(buf, "_ pid=[%d]", getpid());
10    execl("/bin/echo", "echo", "Execution", "d'un test exec", buf,
        NULL);
11
12    // Ces lignes ne s'ex cutent que si execl()  choue
13    printf("[%d] - Echec de la fonction execl()\n", getpid());
14    printf("[%d] - Fin du test exec()\n", getpid());
15    exit(0);
16 }

```

Listing 11 – Programme test exec()

Résultat :

```

[1234] - Debut du test exec()
Execution d'un test exec _ pid=[1234]

```

Explication : `execl()` remplace l'image du processus courant. Si elle réussit, le code suivant n'est jamais exécuté.

6.2 Question 2 : Avec /bin/ech

Résultat :

```

[1234] - Debut du test exec()
[1234] - Echec de la fonction execl()
[1234] - Fin du test exec()

```

Explication : `/bin/ech` n'existe pas, donc `execl()` échoue et retourne -1. Le programme continue son exécution.

7 Exercice 6 : fork() et exec()

7.1 Question 1 : Avec execl()

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10        // Processus fils
11        execl("/bin/ps", "ps", "-l", NULL);
12        perror("execl  chou ");
13        exit(1);
14    } else {
15        // Processus p re
16        int status;

```

```

17     wait(&status);
18
19     if (WIFEXITED(status)) {
20         printf("Commande ps termin e avec code: %d\n",
                WEXITSTATUS(status));
21     }
22 }
23 return 0;
24 }

```

Listing 12 – Exécution de ps -l avec execl()

7.2 Question 2 : Avec execv()

```

1 // Remplacer execl() par :
2 char *args[] = {"ps", "-l", NULL};
3 execv("/bin/ps", args);

```

Listing 13 – Utilisation de execv()

8 Exercice 7 : Exécution de ls -l /

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10         // Processus fils ex cute ls
11         printf("Fils (PID=%d) ex cute ls -l /\n", getpid());
12         execl("/bin/ls", "ls", "-l", "/", NULL);
13         perror("execl chou ");
14         exit(1);
15     } else {
16         // Processus p re attend
17         printf("P re (PID=%d) attend la fin du fils\n", getpid());
18
19         int status;
20         wait(&status);
21
22         if (WIFEXITED(status)) {
23             printf("Commande ls termin e avec code: %d\n",
                    WEXITSTATUS(status));
24         }
25     }
26     return 0;

```

9 Conclusion

Ce TP a permis d’explorer les concepts fondamentaux de la gestion des processus sous Linux :

- La création de processus avec `fork()`
- La synchronisation père-fils avec `wait()`
- La gestion des états de terminaison
- Les processus zombies et leur prévention
- Le remplacement d’image processus avec `exec()`

Les connaissances acquises sont essentielles pour développer des applications multi-processus efficaces sous systèmes Unix/Linux.

Annexe : Références

- Manuel Linux : `man fork`, `man exec`, `man wait`
- W. Richard Stevens, "Advanced Programming in the UNIX Environment"
- Documentation GNU C Library